A Formal Specification of `model-provider` (current state: faithful to `model-provider.sh` as built)

Punt Labs

October 10, 2026

1 Purpose and scope

This document models `model-provider.sh` *as it is implemented today.* It is the faithful companion to `model-provider.target.spec.tex`, which models the more robust design we are building toward. Where the two differ, this one describes the code; the target one describes the intent. Each schema here was checked against the script, and every divergence a peer review found between an earlier, idealised draft and the real code is resolved *in favour of the code* below.

The current tool takes one operator selection — a model served by the **Bifrost** gateway (the only gateway it implements) — and configures four AI coding harnesses (Claude Code, codex, pi, opencode). The guarantee it actually delivers is weaker and more specific than "every request serves the selection". First, the *main* request of every harness serves the selection. Second, Claude Code's *subagent* (tier-alias) and *small* requests are routed to a *different, same-provider-family* id resolved from the live catalog — the highest-version model of that tier in the selection's provider family — not to the selection itself; a haiku subagent is meant to run haiku, not to be force-upgraded onto the selection. Third, `verify` probes *only* Claude Code's Anthropic path against Bifrost; on the direct route it is a successful no-op, not a failure.

Preconditions are written explicitly so TTF partition analysis can derive test cases from each operation's input domain.

2 Given sets and enumerations

A model id is an opaque catalog token (e.g. `openrouter/anthropic/claude-opus-4.8`). A provider is the upstream Bifrost routes a model to (e.g. `openrouter`, `anthropic`, `bedrock`) — in a model id it is the segment before the first slash. A canonical id is the base model name Claude Code derives from a model id (e.g. `claude-opus-4-8`).

$[ModelId, Provider, CanonId]$

The four harnesses.

$$Harness ::= claudecode \mid codex \mid pi \mid opencode$$

Each harness issues requests of distinct kinds. *main* is the primary loop; *subagent* is a Claude Code Task-tool spawn (resolved through tier alias configuration); *small* is a background / side model (Claude Code's small-fast model, opencode's title model). Side-requests are first-class: they are exactly the paths that fail silently when only the main model is set.

$$ReqKind ::= main \mid subagent \mid small$$

The route the tool can be in. The current tool implements exactly two: routed through Bifrost, or "direct" (vendor defaults, no gateway). There is no gateway selection — Bifrost is hardcoded — so unlike the target spec there is no *set* of gateways here, only this route flag.

$$Route ::= bifrost \mid direct$$

Authentication modes a Bifrost route may use: a per-host virtual key (*vkauth*), or the operator's own subscription OAuth passed through Bifrost (*subauth*).

$$AuthMode ::= vkauth \mid subauth$$

Booleans. fuzz's mathematical toolkit has no built-in boolean type, so the specification defines one; *yes*/*no* are its values.

$$Bool ::= yes \mid no$$

3 Harness properties

Which harnesses *canonicalize* the configured id before sending it (strip the provider prefix, rewrite the version), rather than routing it verbatim. Only Claude Code does; this is why it needs an override map to send a dotted/prefixed id unaltered on the wire.

$$canonicalizes : \mathbb{P}\, Harness$$

$$canonicalizes = \{claudecode\}$$

The request kinds each harness actually issues.

$$issues : Harness \longrightarrow \mathbb{P}\, ReqKind$$

$$\begin{aligned} issues = \{&claudecode \mapsto \{main, subagent, small\},\\ &codex \mapsto \{main\},\\ &pi \mapsto \{main\},\\ &opencode \mapsto \{main, small\}\} \end{aligned}$$

4 The Bifrost catalog

The tool reads one live catalog from Bifrost: per model id, the provider it routes to. *catalog* is the set of ids Bifrost knows; the operator may select any of them (validation against this set is exactly what `use` does without `--force`). *provOf* gives the upstream provider of a model id.

$$
\begin{array}{|l}
catalog : \mathbb{P}\ ModelId \\
provOf : ModelId \pfun Provider \\
\hline
\mathrm{dom}\ provOf = catalog \\
catalog \neq \emptyset
\end{array}
$$

5 Tier resolution — the crux of the current design

For Claude Code, the subagent and small request kinds are *not* served by the selection. `claude_tier` resolves them, per tier, to the highest-version catalog id of that tier *in the selection's provider family*. *tierId* models this: given the selection, it yields the id a tier slot is set to. It is a same-family id — its provider equals the selection's provider — but it is generally a *different model*. On a non-Claude selection (no Claude tier exists in that family) the slot is omitted; we model that with the selection's own id standing in only to keep the slot total, but the fidelity property we assert is provider-equality, not id-equality.

$$
\begin{array}{|l}
tierId : ModelId \fun ModelId \\
\hline
\forall\, m : ModelId \bullet \\
\quad m \in catalog \Rightarrow \\
\quad\quad (tierId\ m \in catalog \land provOf\ (tierId\ m) = provOf\ m)
\end{array}
$$

The id that reaches the wire for harness h, request kind k, under selection m, is written inline where it is used, as the conditional expression

$$\textbf{if}\ (h = claudecode \land k \neq main)\ \textbf{then}\ tierId\ m\ \textbf{else}\ m.$$

The main kind of every harness, and every kind of a non-canonicalizing harness that has only side-slots pinned to the selection, carries m verbatim; Claude Code's subagent and small kinds carry $tierId\ m$ — a same-family, generally-different id. This is the honest current-state wire target: it does *not* claim the selection reaches every request. It is inlined rather than packaged as a standalone function because a total function over the $Harness \times ReqKind \times ModelId$ product is not finitely enumerable by the model checker, whereas the inline form ranges only over each harness's finite $issues\ h$.

6 Configuration state

sel is the selection; *route* records bifrost (routed) or direct; *auth* the auth mode. *wire* gives, per harness, per issued request kind, the id that will actually travel. *overrides* records, per harness, whether a canonicalization override has been written. The selection must be a catalog id *unless* it was forced; *forced* records that, so a `use --force` of an off-catalog id is representable (the real tool writes it to the block).

Config

$$
\begin{array}{l}
sel : ModelId \\
route : Route \\
auth : AuthMode \\
forced : Bool \\
wire : Harness \pfun (ReqKind \pfun ModelId) \\
overrides : \mathbb{P}\, Harness \\
\hline
forced = no \Rightarrow sel \in catalog
\end{array}
$$

7 Initial state

A fresh repo reads `BIFROST_ROUTE` with a default of `bifrost`, but with no selection and no managed wire — `status` reports `route:  bifrost` with `model: <unset>`. We model the never-used state as direct with an empty wire, matching what the managed block actually contains (no `ANTHROPIC_MODEL` line) before the first `use`; the route-default read-back is a status concern, not a wire state.

Init

$$
\begin{array}{l}
Config' \\
\hline
route' = direct \\
auth' = vkauth \\
forced' = no \\
wire' = \{h : \{claudecode, codex, pi, opencode\} \bullet h \mapsto \emptyset\} \\
overrides' = \emptyset
\end{array}
$$

8 Well-formedness

wire is defined for exactly the four harnesses and, for each, exactly the request kinds it issues; overrides are only ever recorded for canonicalizing harnesses.

WellFormed

Config

$\mathrm{dom}\ wire = \{claudecode, codex, pi, opencode\}$
$(\forall\, h : \mathrm{dom}\ wire \bullet \mathrm{dom}(wire\ h) = issues\ h)$
$overrides \subseteq canonicalizes$

9 The core guarantee (as built)

Routing correctness, current. When routed through Bifrost, every request every harness issues carries the inline wire target — the selection for main kinds, the same-family tier id for Claude Code's side-requests — and Claude Code has the override that stops its main id being re-canonicalized.

RoutingCorrectCur

WellFormed

$route = bifrost \Rightarrow$
$\quad((\forall\, h : \mathrm{dom}\ wire \bullet \forall\, k : \mathrm{dom}(wire\ h) \bullet$
$\quad\quad (wire\ h)\ k = (\textbf{if}\ (h = claudecode \land k \neq main)\ \textbf{then}\ tierId\ sel\ \textbf{else}\ sel)) \land$
$\quad canonicalizes \subseteq overrides)$

Provider fidelity — the real invariant. The honest guarantee the current code makes: every request, main or side, reaches the *selection's provider*, even when a side-request runs a different model in that family. This is weaker than id-fidelity and is exactly what `claude_tier` guarantees by construction (it only ever picks within the selection's family).

ProviderFidelityCur

RoutingCorrectCur

$route = bifrost \Rightarrow$
$\quad(\forall\, h : \mathrm{dom}\ wire \bullet \forall\, k : \mathrm{dom}(wire\ h) \bullet$
$\quad\quad provOf\ ((wire\ h)\ k) = provOf\ sel)$

The forbidden failure. The historical bug: a canonicalizing harness sent the base id, or a side-request fell back to an unroutable built-in whose provider is not the selection's. Named as the predicate the configuration makes unreachable while routed — the negation of the two guarantees above.

SilentMisrouteCur

WellFormed

$route = bifrost$
$((\exists\, h : \mathrm{dom}\ wire \bullet \exists\, k : \mathrm{dom}(wire\ h) \bullet$
$\quad provOf\ ((wire\ h)\ k) \neq provOf\ sel) \lor$
$\quad(\exists\, h : canonicalizes \bullet h \notin overrides))$

10 Shared effect: applying the Bifrost configuration

The effect every selecting / re-applying operation shares: route through Bifrost with selection m, rebuild *wire* so each issued request kind carries the inline wire target (the selection, or its same-family tier id for Claude Code's side-requests), and record overrides for exactly the canonicalizing harnesses. *fwd*? carries the forced flag through to state.

ApplyBifrost

$$
\begin{array}{l}
\Delta Config \\
m? : ModelId \\
fwd? : Bool \\
\hline
sel' = m? \\
route' = bifrost \\
auth' = auth \\
forced' = fwd? \\
\mathrm{dom}\ wire' = \{claudecode, codex, pi, opencode\} \\
(\forall h : \mathrm{dom}\ wire' \bullet \\
\quad wire'\ h = \{k : issues\ h \bullet \\
\quad\quad k \mapsto (\textbf{if } (h = claudecode \land k \neq main) \textbf{ then } tierId\ m? \textbf{ else } m?)\}) \\
overrides' = canonicalizes
\end{array}
$$

11 Operation: `use`

Selecting a model. Input is a model id and a *force* flag (no gateway input — Bifrost is hardcoded). The precondition domain partitions into: (a) valid — id in the live catalog; (b) forced — *force*? = *yes* bypasses the catalog check; (c) refused — id not in catalog and not forced. An id that strips to empty is refused even under force, but that is a lexical guard on the raw argument, below the model layer, so it is noted here in prose rather than as a state schema.

UseOk

$$
\begin{array}{l}
ApplyBifrost \\
force? : Bool \\
\hline
m? \in catalog \\
fwd? = force? \\
force? = no
\end{array}
$$

A forced selection: the id need not be in the catalog, and the forced flag is recorded so *Config* admits the off-catalog *sel*. This partition is *reachable* here (unlike the target spec's idealised version) precisely because *Config* carries *forced*.

UseForced

$$
\begin{array}{l}
ApplyBifrost \\
force? : Bool \\
\hline
force? = yes \\
fwd? = yes
\end{array}
$$

The refusal: id unknown to the catalog and not forced. No state change.

UseRefused

$$
\begin{array}{l}
\Xi Config \\
m? : ModelId \\
force? : Bool \\
err! : \mathbb{Z} \\
\hline
m? \notin catalog \\
force? = no \\
err! = 1
\end{array}
$$

$$Use \mathrel{\widehat=} (UseOk \lor UseForced) \lor UseRefused$$

12 Operation: `auth`

Set the auth mode and *re-apply the current route unchanged*. The real tool passes the current route through (`write_block "$route" ...`): from bifrost it re-applies bifrost; from direct it stays direct. The `require_model_for_bifrost` guard refuses only when the current route is bifrost *and* no selection has ever been made — so `direct` then `auth vk` must succeed and stay direct. We model the selection-made precondition as $sel \in catalog \lor forced = yes$ (a real selection exists).

AuthOkBifrost

$$
\begin{array}{l}
\Delta Config \\
newauth? : AuthMode \\
\hline
route = bifrost \\
(sel \in catalog \lor forced = yes) \\
sel' = sel \\
route' = bifrost \\
auth' = newauth? \\
forced' = forced \\
\mathrm{dom}\ wire' = \{claudecode, codex, pi, opencode\} \\
(\forall h : \mathrm{dom}\ wire' \bullet \\
\quad wire'\ h = \{k : issues\ h \bullet \\
\quad\quad k \mapsto (\textbf{if } (h = claudecode \land k \neq main) \textbf{ then } tierId\ sel \textbf{ else } sel)\}) \\
overrides' = canonicalizes
\end{array}
$$

On the direct route, `auth` changes only the recorded auth mode: the direct branch of `write_block` ignores the model and writes no wire, so no `ANTHROPIC_MODEL` is touched and the guard does not fire.

AuthOkDirect

$$
\begin{array}{l}
\Delta Config \\
newauth? : AuthMode \\
\hline
route = direct \\
sel' = sel \\
route' = direct \\
auth' = newauth? \\
forced' = forced \\
wire' = wire \\
overrides' = overrides
\end{array}
$$

The refusal: current route is bifrost but no selection has ever been made.

AuthRefusedNoModel

$$
\begin{array}{l}
\Xi Config \\
newauth? : AuthMode \\
err! : \mathbb{Z} \\
\hline
route = bifrost \\
sel \notin catalog \\
forced = no \\
err! = 1
\end{array}
$$

$$Auth \mathrel{\widehat=} (AuthOkBifrost \lor AuthOkDirect) \lor AuthRefusedNoModel$$

13 Operations: `bifrost`, `direct`, `toggle`

GatewayOn (the `bifrost` command) re-applies the current selection on Bifrost; it is refused with no selection (the same `require_model_for_bifrost` guard). *Direct* withdraws the managed wire and overrides, retaining *sel*/*auth* so a later *GatewayOn* re-applies them. `toggle` flips the route, applying the same no-selection guard when it would turn Bifrost on.

GatewayOn

$\Delta Config$

$(sel \in catalog \lor forced = yes)$
$sel' = sel$
$route' = bifrost$
$auth' = auth$
$forced' = forced$
$\mathrm{dom}\ wire' = \{claudecode, codex, pi, opencode\}$
$(\forall h : \mathrm{dom}\ wire' \bullet$
$\quad wire'\ h = \{k : issues\ h \bullet$
$\quad\quad k \mapsto (\mathbf{if}\ (h = claudecode \land k \neq main)\ \mathbf{then}\ tierId\ sel\ \mathbf{else}\ sel)\})$
$overrides' = canonicalizes$

Direct

$\Delta Config$

$route' = direct$
$sel' = sel$
$auth' = auth$
$forced' = forced$
$wire' = \{h : \{claudecode, codex, pi, opencode\} \bullet h \mapsto \varnothing\}$
$overrides' = \varnothing$

Toggle

$\Delta Config$

$sel' = sel$
$auth' = auth$
$forced' = forced$
$(route = direct \Rightarrow$
$\quad (route' = bifrost \land (sel \in catalog \lor forced = yes) \land$
$\quad\quad \mathrm{dom}\ wire' = \{claudecode, codex, pi, opencode\} \land$
$\quad\quad (\forall h : \mathrm{dom}\ wire' \bullet$
$\quad\quad\quad wire'\ h = \{k : issues\ h \bullet$
$\quad\quad\quad\quad k \mapsto (\mathbf{if}\ (h = claudecode \land k \neq main)\ \mathbf{then}\ tierId\ sel\ \mathbf{else}\ sel)\}) \land$
$\quad\quad overrides' = canonicalizes))$
$(route = bifrost \Rightarrow$
$\quad (route' = direct \land$
$\quad\quad wire' = \{h : \{claudecode, codex, pi, opencode\} \bullet h \mapsto \varnothing\} \land$
$\quad\quad overrides' = \varnothing))$

14 Read-only operations: `status`, `list`, `verify`

Status reports the selection, route and auth. *List* reports the catalog. *Verify* is scoped to what the real `verify_model` does: it probes *only Claude Code's* Anthropic path against Bifrost. On the direct route it is a successful no-op (the tool prints a note and returns success) — this is *VerifyDirectOk*, not a failure.

Status

$$\Xi Config$$
$$repSel! : ModelId$$
$$repRoute! : Route$$
$$repAuth! : AuthMode$$

$$repSel! = sel$$
$$repRoute! = route$$
$$repAuth! = auth$$

List

$$\Xi Config$$
$$cat! : \mathbb{P}\ ModelId$$

$$cat! = catalog$$

`verify` succeeds when, routed through Bifrost, Claude Code's *main* request carries the selection and the override is present. It deliberately does *not* check codex/pi/opencode (those are separate manual checks in the test plan) and does not assert the selection on Claude Code's side-requests (they carry tier ids). *WellFormed* is included so a missing Claude Code main slot cannot vacuously pass.

VerifyPass

$$\Xi Config$$
$$WellFormed$$
$$ok! : \mathbb{Z}$$

$$route = bifrost$$
$$(wire\ claudecode)\ main = sel$$
$$claudecode \in overrides$$
$$ok! = 1$$

On the direct route, `verify` is a successful no-op: nothing routes through Bifrost, so there is nothing to prove.

VerifyDirectOk

$\Xi Config$
$ok! : \mathbb{Z}$

$route = direct$
$ok! = 1$

The genuine failure: routed, but Claude Code's main request does not carry the selection, or the override is missing.

VerifyFail

$\Xi Config$
$ok! : \mathbb{Z}$

$route = bifrost$
$((wire\ claudecode)\ main \neq sel \lor claudecode \notin overrides)$
$ok! = 0$

$$Verify \mathrel{\widehat=} (VerifyPass \lor VerifyDirectOk) \lor VerifyFail$$

15 The system invariant

Whenever routed through Bifrost, the configuration is well-formed, every request carries the inline wire target, and every request reaches the selection's provider. This is the precise "works as built" property. It holds by construction of the operations (each rebuilds *wire* via the inline target); the theorem below makes that machine-checkable.

SystemOKCur

$WellFormed$

$$\begin{aligned}
&route = bifrost \Rightarrow \\
&\quad ((\forall h : \mathrm{dom}\ wire \bullet \forall k : \mathrm{dom}(wire\ h) \bullet \\
&\qquad (wire\ h)\ k = (\textbf{if } (h = claudecode \land k \neq main) \textbf{ then } tierId\ sel \textbf{ else } sel)) \land \\
&\quad (\forall h : \mathrm{dom}\ wire \bullet \forall k : \mathrm{dom}(wire\ h) \bullet \\
&\qquad provOf\ ((wire\ h)\ k) = provOf\ sel) \land \\
&\quad canonicalizes \subseteq overrides)
\end{aligned}$$

A machine-checkable obligation: every state reached by the shared apply effect satisfies the system invariant. *ApplyBifrost*'s postcondition builds a well-formed *wire* from the inline target with overrides for the canonicalizing harnesses, so the invariant follows. Stating it as a theorem lets `probcli -cbc_assertions` actually check it, rather than relying on prose.

Theorem (ApplyGivesSystemOKCur).

$$\forall\, Config'; m? : ModelId; fwd? : Bool \bullet (\exists\, Config \bullet ApplyBifrost) \Rightarrow (\theta Config' \in \{SystemOKCur \bullet \theta Config\})$$

16 Per-harness on-disk configuration

The abstract *wire* states the guarantee; this section grounds it in each harness's actual files and model slots, including sub-agent / extension slots, as confirmed against the on-disk configs. A model slot holds a concrete id, or *inherit* (unset — the harness falls back to its parent / main model).

$$Slot ::= inherit \mid fixed\langle\!\langle ModelId \rangle\!\rangle$$

A slot *resolves to the selection* when it is *inherit* or *fixed sel*. For Claude Code's tier/small slots the current code does *not* target the selection, so a second predicate, *resolvesFamily*, captures the real guarantee: the slot is *inherit*, or a *fixed* id in the selection's provider family.

$$resolves : ModelId \longrightarrow \mathbb{P}\, Slot$$

$$\forall\, m : ModelId \bullet resolves\, m = \{inherit\} \cup \{s : Slot \mid s = fixed\, m\}$$

$$resolvesFamily : ModelId \longrightarrow \mathbb{P}\, Slot$$

$$\forall\, m : ModelId \bullet resolvesFamily\, m = \{inherit\} \cup \{s : Slot; i : ModelId \mid s = fixed\, i \land i \in catalog \land provOf\, i = provOf\, m \bullet s\}$$

16.1 Claude Code

Config lives in the managed block in *DIR*`/.envrc.local` and `.claude/settings.local.json` (`modelOverrides`). Slots: *ccMain* (`ANTHROPIC_MODEL`); the four tier-alias slots *ccOpus*/*ccSonnet*/*ccHaiku*/*ccFable* (`ANTHROPIC_DEFAULT_{OPUS,SONNET,HAIKU,FABLE}_MODEL`); and *ccSmall* (`ANTHROPIC_SMALL_FAST_MODEL`). *ccOverride* records the `modelOverrides` map (canonical-key and raw-identity entries).

$$\begin{array}{l} ClaudeCodeConfig \\ \hline ccMain, ccOpus, ccSonnet, ccHaiku, ccFable, ccSmall : Slot \\ ccOverride : Bool \\ \hline \end{array}$$

Claude Code is correct, as built, when the *main* slot resolves to the selection and the override is present, and each tier/small slot resolves *within the selection's*

family (it is set to a same-family tier id, or left to inherit). This is the faithful `ClaudeCodeOK`: it does not demand the tier slots equal the selection, because the code does not set them that way.

ClaudeCodeOKCur

ClaudeCodeConfig
sel : *ModelId*

---

ccMain ∈ *resolves sel*
ccOpus ∈ *resolvesFamily sel* ∧ *ccSonnet* ∈ *resolvesFamily sel*
ccHaiku ∈ *resolvesFamily sel* ∧ *ccFable* ∈ *resolvesFamily sel*
ccSmall ∈ *resolvesFamily sel*
ccOverride = *yes*

16.2 codex

Config lives in `~/.codex/config.toml`: *cxMain* (`model`) and the sub-agent slot *cxSubagent* (`[agents] agent_default_subagent_model`). A spawned sub-agent that omits a model inherits *cxMain*; model-provider leaves it unset, so *inherit* is both default and safe.

CodexConfig

cxMain : *Slot*
cxSubagent : *Slot*

CodexOKCur

CodexConfig
sel : *ModelId*

---

cxMain ∈ *resolves sel*
cxSubagent ∈ *resolves sel*

16.3 pi

Config lives in `~/.pi/agent/settings.json` (*piMain*, `defaultModel`); sub-agents are Markdown files whose omitted frontmatter `model:` inherits the session model. *piSubagent* models that slot; *inherit* is the safe default.

PiConfig

piMain : *Slot*
piSubagent : *Slot*

PiOKCur

PiConfig
sel : *ModelId*

---

piMain ∈ *resolves sel*
piSubagent ∈ *resolves sel*

16.4 opencode

Config lives in `~/.config/opencode/opencode.json`: *ocMain* (`model`), *ocSmall* (`small_model`), and per-agent slots *ocAgent* (`agent.<name>.model`). The current tool pins *ocSmall* to the selection (`small_model = openai/<model>`, proven routable), so for opencode even the small slot resolves to the selection — unlike Claude Code. An agent that omits `model` inherits the session model.

OpencodeConfig

ocMain : *Slot*
ocSmall : *Slot*
ocAgent : *Slot*

OpencodeOKCur

OpencodeConfig
sel : *ModelId*

---

ocMain ∈ *resolves sel*
ocSmall ∈ *resolves sel*
ocAgent ∈ *resolves sel*

16.5 The composed fleet invariant

When routed, Claude Code is correct to *family* fidelity on its side-slots and *id* fidelity on main; codex, pi and opencode are correct to id fidelity. This is the file-level, sub-agent-inclusive restatement of *SystemOKCur*.

Fleet

sel : *ModelId*
route : *Route*
ClaudeCodeConfig
CodexConfig
PiConfig
OpencodeConfig

$$
\begin{array}{|l}
\textit{FleetOKCur} \\
\textit{Fleet} \\
\hline
\textit{route} = \textit{bifrost} \Rightarrow \\
\quad (\textit{ClaudeCodeOKCur} \land \textit{CodexOKCur} \land \textit{PiOKCur} \land \textit{OpencodeOKCur}) \\
\hline
\end{array}
$$

17 What this spec does not claim

For honesty, the current-state limitations modelled or noted above are as follows. Only Bifrost is implemented; there is no gateway selection. Entitlement is not enforced: the catalog domain is the whole selectable set, so a catalogued-but-unentitled id (the Bedrock Anthropic-models gap) can be selected and routed — the target spec closes this. `verify` proves only Claude Code's main Anthropic path; the other harnesses and side-requests are verified by the separate manual procedure in the test plan. Finally, `direct` leaves some opencode provider/model residue in place (a documented cleanup gap); *Direct* here idealises that to an empty wire — the one place this spec is slightly kinder than the code, called out rather than hidden.

18 Two real-world constraints: compatibility and credential terms

Two real-world facts bound correct operation; neither is enforced by the current tool, so both are documented gaps.

Compatibility (support, not permission). Anthropic does *not* support routing Claude Code to non-Claude models. This is a support and compatibility statement, not a terms violation: with a non-Anthropic selection on the Claude Code path, tool use, extended thinking, and prompt caching are unvalidated and degrade per model, so routing fidelity is necessary but not sufficient. Separately, two Claude Code defaults — MCP tool search and fine-grained tool streaming — silently turn off whenever Claude Code is routed through the gateway (the `bifrost` route), regardless of model; both are recoverable via environment variables. The current tool does not restore them.

Credential terms. The licensing line runs along the credential, not the model: a Claude Pro/Max subscription is licensed for Anthropic's own clients (the Claude apps and Claude Code), so spending it on a third-party client is the violation — for *any* model, Claude included. In this model *subauth* is the subscription-OAuth mode, legitimate only on the Claude Code path. The current tool does *not* guard this: it passes *auth* through ($vk \mid subscription$) without checking that a subscription credential only backs Claude Code, so a misconfiguration could point a subscription at a non-Anthropic harness. The target spec closes this with an enforced precondition; here it is a known gap.